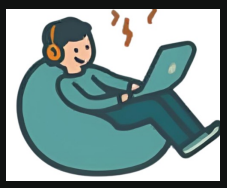

The Lazy Developer

04

MODULE

Debugging: Painful, mandatory, yet strangely rewarding.

Patience is more than a virtue, it's mandatory! How to keep your optimism and focus while trying to debug various errors.

Table of Contents

01. Why debugging is worth your time.

8 min

Why Bugs Happen (and How You'll Spot Them)

Debugging is basically tech world sleuthing. /Think of it like noticing your smart bulb never turns purple on command: you know something's wrong, you just don't know where. / You will immediately find yourself debugging when you start using Cursor. I will be genuinely impressed if you manage to fire up your dev environment without a single hiccup! If you that, you're going to really excel in this space of AI-assisted programming!

What a Bug Looks Like in Real Life:

You don't need to read code to notice a bug; you just need to notice weirdness:

- Blank or frozen screens – The web page half loads, then stalls like a food truck that ran out of propane.
- Crash and burn moments – Tap Share in an app and… /poof, it closes.
- Wrong answers – A "20 /% off" coupon gives you $0.13 instead of $2.60.
- Mysterious javascript console pop ups – A red text message flashes at the bottom of your browser tools saying something like TypeError: cannot read property 'map' of undefined.

Those red text clues are pure gold—they're the software's way of yelling, "Here's what hurt!"

Error Flavours Across Popular Languages

Different programming languages have different "accents" when they complain. Here's a quick sampler of things where you'll see the tone and style change, even if the underlying issue is the same:

Language (Typical Use)	How It Alerts You	What the Message Might Say	When You'd Notice
TypeScript (modern web apps)	Stops you before you even run the page.	Property 'trim' does not exist on type 'number'.	While the developer is saving files or during the build step.
JavaScript (web everywhere)	Lets you run, then throws red text in the browser console.	Uncaught ReferenceError: total is not defined	When you open DevTools or the page refuses to behave.
Python (data scripts, automations)	Prints a "traceback" list ending with the exact error name.	FileNotFoundError: [Errno 2] No such file or directory: 'sales.csv'	In the terminal window that launched the script.
Rust (systems, high performance)	Refuses to compile until every possibility is handled.	error[E0382]: use of moved value: 'file_handle'	At compile time; code literally won't build until fixed.
Ruby (automation, web back end)	Shows a colourful stack trace with file names.	undefined method 'size' for nil:NilClass	In the terminal or the web server logs.

> *Some languages are "helicopter parents" who won't let you leave the house with mismatched socks; others let you run outside, skin your knee, then teach you a lesson. Both styles have pros and cons—you just need to read their love notes (error messages).*

1. Read top to bottom, bottom to top, or both – Error logs often include a "stack trace," a breadcrumb trail of function names. The last line usually tells you the exact complaint with a line number from the file in question the AI will use to find exactly where the issue starts.
2. Spot the nouns – File names, variable names, line numbers. Those are your map coordinates.

3. Don’t panic at jargon – Words like undefined, null, or panic! sound dramatic but often mean “Hey, we expected something here and got nothing.”

Claude, Gemini, and ChatGPT are able to help—provided you give them the right clues.

Give the LLM…	…Because	Example Prompt
The full error message (copy paste it)	Tiny wording differences matter.	“Here’s the Python error I saw: ValueError: could not convert string to float: '€10'. What usually causes this?”
What you expected to happen	Helps the model reason about intent.	“I was converting prices so I could add them up.”
A snippet of surrounding code or description of the step	Context prevents wild guesses.	“The value comes from a CSV column that sometimes includes the euro symbol.”
Environment details (browser, OS, language version)	Some bugs are version specific.	“Running Python 3.12 on macOS Sonoma.”
What you tried already	Avoids suggestions you’ve ruled out; speeds new ideas.	“I already removed commas and spaces, still breaks.”

Quick Fire Tips for Prompting (go back to module 1 and 2 to get a refresher on this).

- Be specific – “My web page is blank” !’ “The React component shows a white screen and the console says: TypeError: Cannot read property 'map' of undefined.”
- Chunk the problem – Start with the single error, then zoom out if needed.
- Ask for next steps – “What’s one experiment I can run to confirm the source?”

Putting It All Together: A Mini Scenario

> *Scenario: You click Checkout on a demo web store. A spinner appears…then nothing. Opening the browser console you read:*
> *Uncaught TypeError: discount.toFixed is not a function*

So, what can you tell the AI?

1. Observe – Spinner stuck, console error points to discount.
2. Copy the clue – Grab the full red line.
3. Ask in the chat window –“I’m testing a shopping cart demo. Console shows (attach screenshot, or copy the text) Uncaught TypeError: discount.toFixed is not a function at Prices.js:42. I expected the price field to show dollars and cents. Running Chrome on Windows 11. Any ideas why toFixed might fail?”
4. Act on suggestions – Maybe the model tells you discount is coming back as a string like "5%", which can’t use numerical methods. It will then make a modification to that file (orange diff remember?).
5. Verify fix – Test if you can while the file is in memory as changed. Change is (parseFloat(discount)), reload, run your same workflow again, ah! The spinner disappears, price looks right. Victory dance ensues.

Key Takeaways Before We Move On

- Bugs show symptoms long before you peek at code—learn to notice the oddities.
- Error messages are treasure maps. Even cryptic ones contain X marks the spot clues.
- Different languages, same detective game—just different accents.
- LLMs work best with evidence. Feed them the full message, context, and your hypothesis, and they'll return practical next moves.

02. Bring your patience (and your confidence)

12 min

Why “Something Broke” Feels So Personal

When a web form refuses to submit or an email campaign won’t send, it’s easy to think, “I did something wrong.” Your heart rate jumps, shoulders tense, and suddenly every click feels risky. That spike is stress chemistry, and it makes careful thinking harder. Before you copy one more setting or mash Refresh, pause long enough to calm the circuitry that’s yelling fix it now!

> *Some bugs will be fixed in one change, some are going to drive you batty enough to go eat something, take a walk, have a shower, or whatever mental break you think need. I'm been through this so many times since starting my journey. I think my record for a bug hold out was about 10 hours of work spread over 3 days. It required me to move from Vercel to Railway to get things fully working the way I wanted to. It didn't mean that it couldn't be fixed running entirely on Vercel, it was going to mean a lot of code refactoring (updating) to make things work. Or, I could make small changes and move the whole thing to Railway and not have to worry so much.*

These days, AI has gotten SO MUCH BETTER! The ability for it to resolve issues is far lower, especially if you give it the right context (remember that?).

Think of the AI (ChatGPT, Claude, Gemini or whatever tool you use) as a patient tutor who can look up manuals faster than you can type a URL. It can’t help, though, if it doesn’t know what you’re seeing. So treat the conversation like a tech support ticket:

1. Describe the symptom in plain language. “The newsletter test sent but the links in the email are blank squares.”
2. Paste any error pop ups or log lines you can copy. They’re clues, even if they look like gibberish.
3. Say what you expected so the AI knows the goal.
4. Tell it your environment (Chrome on Windows /11, Zapier free plan, etc.) - if you haven't already done so.
5. Enable extra console logging to give you more red error lines in the browser to trace more activity. Just ask the AI to add the code.

If the AI’s first answer misses the mark, nudge it:

> *“That didn’t solve it. Here's the error message.” Optionally, if the error is exactly the same, "I have rolled back your change as a result" and that's when you click 'reject' in the code window.*

It's going to try again, and again, and again. So, if you're going to go down that route, keep giving it context each time, roll back if you can, enable logging for more data, and see where you get.

> *Errors can change, so don't just look for a failure.*

Moving from a 404 > 400 error is progress. Or 403 > 500 is also progress! It means you've resolved that last error and now here's the next one to pass. Keeping the AI up to date on these changes are critical to getting through errors.

I'm going to keep drilling this concept into your minds but presented in different ways because it needs to become muscle memory if you want to develop apps fully.

What to Share	How to Obtain It	Why It Helps
Exact error text	Copy paste from the pop up, console, or log file.	AI can search the web or its training to find known fixes.
Relevant screenshots	Attach or paste images (redacted if needed).	Visual context catches things wording misses.
Links to docs or support pages	Grab the URL of the tool's SDK/ API docs or FAQ.	Tools change; fresh docs let the AI reason with up to date rules.
Your last change	"I switched the API key scope from 'read' to 'write' yesterday."	Narrows the suspect list quickly.

When Third Party Integrations Misbehave

Most "mystery failures" today involve stitching services together: CRM !' email platform, form !' Google /Sheets, payment gateway !' website.
Here's a repeatable playbook:

1. Find the official docs
   Search: "<tool name> API docs" or "<tool> SDK".
2. Share the link with the AI"Here's the SendFast API changelog from May /2025: [URL]. Can you spot anything that could break a simple 'send email' call?"
3. Ask for alignment"Given the doc above, is my request body missing a required field?"

Because the AI can read and reason over the page you linked, it often uncovers mismatches you'd spend hours eyeballing. If you do your own Google searching in addition to the AI, share your results too! It will tell you if that's relevant or not. You DON'T need to read if you don't want to. Let the AI do that and explain to you what was useful or not useful.

Key Takeaways

- Calm first, clicks second. Stress clouds judgement; a 30 second pause is free.
- Treat AI like a teammate: feed it symptoms, context, and links so it can think with you.
- No coder? No problem. Error text + official docs + focused prompts beat random guess and check.
- Integrations evolve. Always pull the latest SDK/API docs and hand them to your AI for comparison.
- Time box loops. If one idea stalls, pivot: new prompt, new search, or new doc.

03. A simple, repeatable, debugging game plan.

10 min

A Simple, Repeatable Debugging Game Plan
(the four step loop you'll run always with an "undo /button" close at hand)

Before we start, picture a toddler stacking blocks. She adds one at a time, checks that the tower still stands, and—if it wobbles—removes the last block before trying again. Debugging works the same way: tiny, reversible moves let you learn quickly without turning the tower into rubble.

Step /1 /– /Reproduce
Goal: make the problem appear whenever you want, as if you had a "Replay Bug" button.

1. Write down the exact steps (click "Submit", choose "Blue", press Save).
2. Record the symptom (blank page, 404 error, prices all zero).
3. Hand those steps to the AI."If you do X /!' /Y /!' /Z, the checkout freezes at the spinner." Now the AI can think alongside you instead of guessing.

Why it matters: If you can't trigger the bug on demand, you can't be sure you ever fixed it.

Step /2 /– enable lots of logging
Goal: generate lots of noise to see all the associated activity to get a clearer picture of what's going on under the hood.

Ask the AI to enter code to enable console and server logging. This will give you far more information to pass when you re-run the error with the logging enabled.

Why it matters: You are almost certain to uncover new activities that you didn't quite realise, or simply validate that it is not something else that is the issue. Then once it's all done you can turn off all that extra logging (as it's a waste of resources and just noise).

Step /3 /– /Guess /& /Test
Goal: form one small hunch, change one small thing, watch the result.

Smart Move	Why	AI Prompt Example
Make only one change at a time	Confirms cause and effect.	"I'm switching the 'from' email to a verified address—nothing else."
Snapshot or duplicate before editing	Gives you a quick undo.	"Show me how to clone this workflow so I can test safely."
Roll back fast if nothing changes or a new error appears	Keeps the playing field clean; avoids stacking confusion.	"Revert last change—now what's the original error again?"

Rolling back: the safety net
Think of rollback as a Cmd-Z or Ctrl Z for your whole experiment. If a tweak doesn't help—or trades one error for two—undo it immediately unless the only thing you added was extra logging. Staying rolled forward can:

1. Hide the real problem behind fresh ones.
2. Waste AI suggestions on side effects you just created.
3. Leave messy breadcrumbs you'll forget to clean up.

So: test !' evaluate !' revert !' plan next move.

Step /4 /– /Lock /It /In
Goal: once it works, freeze the victory and record the lesson.

1. Take one final snapshot (copy of the flow, git commit, zipped folder).
2. Write a two line summary in plain language:Fixed 400 error by verifying 'from' address.
   Root cause: service now rejects un verified senders (change on 2025 04 15).
3. Ask the AI to draft a future proof note or automated test:"Generate a weekly check that sends a test email and alerts me if it fails."

Locked in knowledge pays dividends: tomorrow you (or a teammate) will search the chat, find the note, and skip straight to the winning move. Remember to turn off that extra logging too!

Putting the Loop Together—A Quick Walk Through

> *Bug: Your payment form shows "Invalid card" for every Visa number.*
>
> *Reproduce: You note: Enter any Visa card !' click Pay !' error banner appears. AI confirms it can't complete the flow either.*
>
> *Shrink: You disable coupon code field, shipping selector, and try the form in a sandbox mode—still fails, so the issue is inside the payment step.*
>
> *Guess /& /Test: Hunch: the integration is in test mode, but Visa test numbers changed. You swap in the new sample number from Stripe docs. The error message changes to "Card expired." You roll back—wrong hunch. Next guess: the "Allowed card types" list is empty. You add "Visa", run again—payment succeeds. Rollback? Not this time; success means keep the change.*
>
> *Lock /It /In: You snapshot the working settings, ask the AI to add a monthly task that pings the payment API with test cards, write a note in the project wiki, and call it done.*

Key Points to Remember

- Reproduce, Shrink, Guess /& /Test, Lock /It /In—that's the loop.
- One change at a time, undo at the first sign of noise.
- Rollbacks keep the crime scene intact; only skip them when adding harmless logs.
- Document the fix and set an automated check so the bug can't sneak back.

- Your AI partner is there at every step; feed it snapshots, docs, and results, and it will keep the investigation on track.

04. Leave breadcrumbs for the next explorer

8 min

Effective debugging does not end when the error disappears. Unless the underlying rationale is captured, the same issue or a closely related one will eventually return. The following practices create a durable trail of evidence that future maintainers, auditors, or even your own future self can follow without repeating the entire investigation.

Inline Explanations: “Why”, Not Merely “What”

Whenever you adjust a script, workflow, or configuration file, have the AI annotate the change in plain language at the point of alteration by adding comments into the code.

- Focus on intent.“Stripe now rejects unverified ‘from’ addresses (policy change /2025 04 15). Verified ‘billing@example.com’ to comply.”
- Keep the wording neutral and concise. Avoid informal remarks such as “quick hack” or “temporary fix,” which provide no actionable context.
- Update or remove obsolete notes. Stale comments create confusion and dilute trust in the documentation.

/Structured Commit or Change Messages

Many tools like Git repositories, low code platforms, or SaaS workflow builders maintain automatic version histories. Each saved revision should be accompanied by a clear, purpose driven message.

Component	Recommendation	Example
Summary line	Imperative verb + concise object /("d /50 /characters).	“Verify sender domain for emails”
Detail paragraph	Rationale, scope, and reference IDs (tickets, docs).	“Email service updated policy on 2025 04 15. Unverified ‘from’ addresses now trigger HTTP /400. Added DNS verification and updated 2 upstream steps, marking verified address.”
AI assistance	Provide the diff or list of edits and request a compliant message.	“Draft a commit message explaining the policy change and the fix; limit summary to 50 characters.”

A well structured commit history becomes a chronological narrative of debugging decisions, making audits and rollbacks straightforward.

Maintaining a Debug /Diary (more useful in massive codebases with multiple people working on it)

For complex investigations, or adding major features, those requiring multiple hypotheses, external research, or cross team collaboration, maintain a short “debug diary” in a shared knowledge base (wiki, Notion, Confluence).

1. Timestamp each entry. Chronology helps reconstruct the decision path.
2. Record observations, discarded ideas, and turning points. Even failed experiments prevent colleagues from revisiting the same dead ends.
3. Link to external artifacts. Include URLs to SDK documentation, support tickets, or AI chat excerpts that informed the resolution.

4. Summarise the outcome. Close each incident with root cause, remediation, and—if applicable—monitoring steps to prevent recurrence.

AI as a diary assistant
After a fix is confirmed, paste the chat transcript or investigation notes and request:

> *“Condense this conversation into a post mortem entry: symptom, root cause, fix, and future guardrails.”*

The model can generate a polished, uniform report in seconds, ensuring the diary remains readable and valuable. The fix for this can then be entered into your changelog referencing your wiki file.

Consequences of Skipping Documentation

Don't be lazy and don't be stupid! TheLazyDeveloper is about being efficient! You will not remember everything you did and having something that explains it will save you so much time in the future. Plus, there's no reason to be lazy when AI is writing everything for you anyway!

Otherwise, you will have all kinds of issues such as:

- Knowledge attrition: Team members change roles; undocumented reasoning disappears with them.
- Repeated outages: An unrecorded workaround may break again after routine updates.
- Audit and compliance risk: Regulated industries require traceability for data handling changes.
- Inefficient AI assistance: When history is missing, future prompts become longer and less precise, increasing resolution time.

Think long term - what if you want to sell this app you've built. How are the new owners going to run it? What if you're going to sell a packaged version to many people. They need documentation to install, setup, and use it to share amongst their teams.

05. Additional hacks that will make your life easier

10 min

Here's some other items you can use to your advantage in your debugging journey beyond a Google search or looking at Stack Overflow.

Dashboards, Logs and Real Time Telemetry

Application dashboards and log streams serve the same purpose as a heart rate monitor during exercise: they reveal live, continuous behaviour that may not trigger a formal error. Typical metrics include request counts, error rates, response times and user events.

Have the AI build you a dashboard page that monitors specific things you think are important in your app:

Example 1: I have an app that runs a background process that calls an API from Apify. One of the key metrics I use is how long it takes for this process to run as it's normally under 45 seconds. Any runs that go over 60 seconds are marked for me.

Example 2: The same app also stores video data for analysis temporarily. So I have a dashbaord that studies the storage so that I'm not going overboard on my cloud storage costs.

Debugging in Production Safely

Build yourself a "Maintenance Button"

All my apps have an "Enable Maintenance" button in an admin dashboard. This allows me to flip the website into a service mode so nobody can log in while I continue to work under the hood. Then, when I've got everything working, I can turn that feature off and the site is alive again serving users. Normally, you would do this during "planned downtime" but sometimes you won't have that luxury but you need to stop people from using the live site so you can just test all is working fine.

When an incident affects users, fixes often need to be validated in an environment that mirrors production while keeping real customers unaffected. This is your staging environment.

Recommended workflow

1. Reproduce in development.
2. Promote to a staging or preview environment that mirrors live configuration and data shape. This could be a 'staging' branch in your Github that syncs with a staging deployment or a preview build.
3. Run tests and spot checks in staging to make sure all works fine. If it doesn't, go back to dev and start again. You're not affecting or disrupting production.
4. Perform a limited or “canary” release to a small user slice; monitor dashboards for regressions.

AI assistance
Ask the model to generate a roll back plan or deployment checklist so the release can be reversed quickly if unexpected side effects surface.

When You Need a Second Opinion on Code

You may occasionally generate code via the AI that isn't quite working right and you're not sure you're advancing in your debugging. Several web services translate such fragments into plain English that you couldn't then use to go back to the AI and potentially ask new questions knowing what the code says, or you can nowadays try another LLM!

Service	Primary Function
CodeRabbit — AI powered pull request reviewer that produces context aware summaries and suggestions.	WhatDoesThisCodeDo.com — paste any snippet and receive a paragraph level explanation.	AI Code Explainer (zzzcode.ai) — free online tool that clarifies logic in multiple languages.

Upload or paste the snippet, review the explanation, then forward both code and explanation to your main AI assistant for integration advice.

Practical Sequence for Non Coders

1. Attempt to reproduce /!’ /set a breakpoint if the platform allows it.
2. Activate verbose logging and capture a five minute window.
3. Consult dashboards for correlated spikes.
4. Do web searches, share API docs
5. If unclear, submit the implicated code to an online explainer, then feed both to your AI assistant.
6. Apply the fix in development, validate in staging, monitor in production.

Following this structured progression ensures that each investigative activity adds new information, minimising trial and error cycles.